

Class_6_SQL_All_Definers

1. fullname

Name: The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE DEFINER='root'@'localhost' FUNCTION `fullname` (f varchar(20),l varchar(20))
2   RETURNS varchar(50) CHARSET utf8mb4
3   BEGIN
4
5   RETURN concat(f," ",l);
6   END
```

2. age

Name: The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE DEFINER='root'@'localhost' FUNCTION `age` (d date) RETURNS int
2   BEGIN
3
4   RETURN timestampdiff(year,d,curdate());
5   END
```

3. net_salary

Name: The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE DEFINER='root'@'localhost' FUNCTION `net_salary` (s int)
2   RETURNS int
3   BEGIN
4
5   RETURN 0.1*s+s;
6   END
```

4. get_title

Name: The name of the routine is parsed automatically from the DDL statement. The DDL is parsed automatically while you type.

DDL:

```
1 • CREATE DEFINER='root'@'localhost' FUNCTION `get_title`(g char(1), e varchar(30))
2 RETURNS varchar(30) CHARSET utf8mb4
3 BEGIN
4 if g="M" then
5     return concat("Mr.",e);
6 else
7     return concat("Ms.",e);
8 end if;
9 END
```

5. std1_BEFORE_INSERT

▼ BEFORE INSERT
std1_BEFORE_INSERT
AFTER INSERT
BEFORE UPDATE
AFTER UPDATE
BEFORE DELETE
AFTER DELETE

```
1 • CREATE DEFINER='root'@'localhost' TRIGGER `std1_BEFORE_INSERT` BEFORE
2 if new.age<18 then
3     set new.age=18;
4 end if;
5
6 END
```

6. std1_AFTER_INSERT

▼ BEFORE INSERT
std1_BEFORE_INSERT
▼ AFTER INSERT
std1_AFTER_INSERT
BEFORE UPDATE
AFTER UPDATE
BEFORE DELETE
AFTER DELETE

```
1 • CREATE DEFINER='root'@'localhost' TRIGGER `std1_AFTER_INSERT` AFTER IN
2 insert into audit(actions,sid,sname,new_age) values
3 ("Ins",new.sid,new.sname,new.age);
4 END
```

7. std1_BEFORE_UPDATE

The screenshot shows the MySQL Workbench interface. On the left, a tree view lists various triggers, with 'std1_BEFORE_UPDATE' selected under the 'BEFORE UPDATE' category. The main editor displays the following SQL code:

```
1 CREATE DEFINER='root'@'localhost' TRIGGER `std1_BEFORE_UPDATE` BEFORE
2   if new.age<18 then
3     set new.age = old.age;
4   end if;
5   END
```

8. std1_AFTER_UPDATE

The screenshot shows the MySQL Workbench interface. On the left, a tree view lists various triggers, with 'std1_AFTER_UPDATE' selected under the 'AFTER UPDATE' category. The main editor displays the following SQL code:

```
1 CREATE DEFINER='root'@'localhost' TRIGGER `std1_AFTER_UPDATE` AFTER UP
2
3   insert into audit(actions,sid,sname,old_age,new_age) values
4   ("Upd",old.sid, old.sname, old.age,new.age);
5   END
```

9. std1_BEFORE_DELETE

The screenshot shows the MySQL Workbench interface. On the left, a tree view lists various triggers, with 'std1_BEFORE_DELETE' selected under the 'BEFORE DELETE' category. The main editor displays the following SQL code:

```
1 CREATE DEFINER='root'@'localhost' TRIGGER `std1_BEFORE_DELETE` BEFORE
2   if old.dept = "IT" then
3     signal sqlstate '45000'
4     set message_text = "IT Student cannot be removed";
5   end if;
6   END
```

10. std1_AFTER_DELETE

The screenshot shows the MySQL Workbench interface. On the left, a tree view lists various triggers, with 'std1_AFTER_DELETE' selected under the 'AFTER DELETE' category. The main editor displays the following SQL code:

```
1 CREATE DEFINER='root'@'localhost' TRIGGER `std1_AFTER_DELETE` AFTER DE
2   insert into audit(actions,sid,sname,old_age) values
3   ("Del",old.sid,old.sname,old.age);
4   END
```